Doc Simulator

April 8, 2026

0.1 Assumptions

- Replenishment missions are not considered; hence, inventory constraints are neglected. Each pod contains a limited set of SKUs, but the exact quantity of each SKU or its volume are not known.
- Technical aspects related to robots, such as energy management and routing, are out of scope and therefore not modeled.

1 Overview

Il simulatore ad eventi discreti emula la dinamica operativa di un RMFS (Robotic Mobile Fulfillment System) in stile Amazon, basato su un approccio parts-to-picker, modellando il flusso degli ordini (dall'arrivo al completamento) e la movimentazione di robot e pod nel sistema.

Il progetto supporta due modalità decisionali: (i) un approccio a ottimizzazione ciclica, in cui ogni Δt_{opt} viene risolto un modello per la generazione e assegnazione dei task; (ii) un approccio reattivo, basato su policy con lookahead limitato, utilizzate per prendere decisioni online al verificarsi degli eventi.

1.1 Project Structure

Il codice è organizzato su tre livelli logici:

Path	**Role**
run_simulation.py	Entry point: configures logging, builds warehouse, runs simulation
config.py	All numeric parameters in one place
scripts/core/enums.py	Enumerations for all discrete states and event types
scripts/core/entities.py	Domain dataclasses: `Order`, `Pod`, `Robot`, `Workstation`, ...
scripts/core/queues.py	Generic priority queue (min-heap with O(1) id lookup)
scripts/core/warehouse.py	Physical layout, enity generation, distance utilities
scripts/sim/Simulator.py	Core DES engine: clock, event loop, dispatch table
scripts/sim/event_handler.py	One handler function per event type
scripts/opt/policies.py	Heuristic assignment policies

2 Input Information

La tabella seguente riassume i principali parametri del simulatore, indicando per ciascuno il dominio di valori considerato e alcune informazioni aggiuntive sul loro ruolo o utilizzo.

[1] Ancora da definire; dove presente, il valore è quello usato da Barnhart o fissato a priori.
[2] Parametro preso dalla repository GitHub `https://github.com/alexschmid3/warehouse-task-assignment`.

Parametro	**Dominio**[1]	**Info**
`NUM_PODS`	???	Numero totale di pod (`GRID_ROWS` × `GRID_COLS`)
`NUM_SKUS`	???	Numero di tipi di SKU distinti
`NUM_ROBOTS`	???	Numero di robot mobili autonomi
`NUM_WORKSTATIONS`	???	Numero di postazioni di picking
`GRID_ROWS / GRID_COLS`	??? / ???	Dimensioni della griglia di stoccaggio pod
`WS_ORDER_CAPACITY`[2]	3	Numero massimo di ordini aperti per postazione
`WS_WORKLOAD_CAPACITY`	???	Numero di missioni rilasciate per workstation
`ROBOT_SPEED`	30.0	Velocità del robot in celle/min (≈ 0,5 m/s)
`POD_PROCESS_TIME`[2]	5 sec	Tempo di processo di un pod alla workstation
`ITEM_PROCESS_TIME`[2]	5 sec	Tempo di picking di un item alla workstation
`INTERARRIVAL_TIME_ORDER`[2]	$\left\{\frac{12}{35}, \frac{6}{25}, \frac{3}{10}, \frac{3}{40}, \frac{1}{5}, \frac{3}{50}, \frac{3}{100}, \frac{3}{250}\right\}$	Tempo medio (in min) di interarrivo ordini
`PROB_1_ITEM_ORDER`[2]	0.5	Probabilità che un ordine contenga un solo articolo
`GEO_DIST_PARAM_ORDER`[2]	{0.25, 0.35, 0.45, 0.65}	Parametro geometrica per ordini multi-articolo
`TIME_HORIZON`	???	Durata totale della simulazione (in min)
`DELTA_T_OPT`[2]	15 min	Intervallo di chiamata dell'ottimizzatore

Table 1: Parametri di configurazione della simulazione.

3 Domain Entities

Le entità del simulatore sono implementate come dataclass Python. Di seguito vengono descritti i campi principali di ciascuna.

3.1 Order

Rappresenta un ordine cliente lungo tutto il suo ciclo di vita.

Campo	Tipo	Descrizione
`order_id`	`int`	Identificatore univoco dell'ordine
`arrival_time`	`float`	Tempo di simulazione all'arrivo dell'ordine
`order_size`	`int`	Numero totale di SKU richieste dall'ordine
`items_required`	`list[int]`	Lista completa delle SKU richieste
`items_pending`	`list[int]`	Item non ancora prelevati (decrementata durante l'esecuzione)
`workstation_id`	`int | None`	Workstation assegnata dall'ottimizzatore (`None` se non ancora assegnato)
`status`	`OrderStatus`	Stato corrente del ciclo di vita (`BACKLOG`, `WAITING`, `OPEN` o `CLOSED`)

3.2 Pod

Rappresenta un pod di stoccaggio che può essere trasportato da un robot.

Campo	Tipo	Descrizione
`pod_id`	`int`	Identificatore univoco del pod
`storage_location`	`tuple[int,int]`	Cella di stoccaggio nella griglia quando il pod è idle
`items`	`list[int]`	Item (SKU) attualmente contenuti nel pod
`status`	`PodStatus`	`IDLE` (a riposo) o `BUSY` (in transito o in attesa)

3.3 Robot

Rappresenta un robot mobile autonomo che preleva e trasporta i pod.

Campo	Tipo	Descrizione
`robot_id`	`int`	Identificatore univoco del robot
`position`	`tuple[int,int]`	Posizione corrente nella griglia
`current_task_id`	`int | None`	Task in esecuzione (`None` se idle)
`status`	`RobotStatus`	`IDLE` o `BUSY`

3.4 Workstation

Modella lo stato di una postazione di picking operata da un operatore umano.

Campo	Tipo	Descrizione
`workstation_id`	`int`	Identificatore univoco della postazione
`order_capacity`	`int`	Numero massimo di ordini attivi simultaneamente
`workload_capacity`	`int`	Numero massimo di task rilasciate per la workstation (indicazione più che hard-constraints)
`position`	`tuple[int,int]`	Coordinate nella griglia
`opened_orders`	`set[int]`	Ordini attualmente in lavorazione
`pod_buffer`	`list[int]`	Pod in attesa (escluso quello attivo)
`order_buffer`	`list[int]`	Ordini in attesa di essere aperti
`pending_tasks`	`list[Visit]`	Task rilasciati ma non ancora allocati a un robot
`active_tasks`	`list[Visit]`	Task correntemente allocati a un robot
`status`	`WorkstationPickingStatus`	`IDLE` o `BUSY` (si riferisce all'azione del picking)

3.5 Visit e Task

Una `Visit` rappresenta una singola fermata all'interno di una missione: specifica la workstation di destinazione, gli ordini serviti (`orders`) e la lista di item da prelevare (`items`).

Un `Task` è una sequenza ordinata di fermate (`stops`) assegnata a una coppia pod-robot. Ogni task porta un campo `priority` utilizzato dalla coda con priorità per determinare l'ordine di dispatching.

3.6 Enumerations

`OrderStatus`, `RobotStatus`, `PodStatus` e `WorkstationPickingStatus` sono oggetti di tipo `Enum` che descrivono in modo univoco gli stati delle entità durante la simulazione. L'uso delle enumerazioni permette di gestire le transizioni di stato in maniera chiara e coerente, semplificando la logica di simulazione e il tracciamento degli eventi nei log e nella raccolta delle statistiche.

4 Warehouse Layout

La classe `Warehouse` organizza e raccoglie le entità fisiche del sistema (`Pod`, `Robot` e `Workstation`). A partire dai parametri di input `grid_rows` e `grid_cols`, viene generata una griglia rettangolare di pod con dimensioni totali:

$$X = \text{grid_rows} + 2\left\lfloor \frac{\text{grid_rows} - 1}{2} \right\rfloor + 2 \cdot \text{MARGIN} - 1, \qquad Y = \text{grid_cols} + 2 \cdot \text{MARGIN} - 1$$

Ogni due colonne di pod viene inserita una strada larga due celle per il passaggio dei robot, mentre i margini laterali consentono la distribuzione delle workstation. Le postazioni di picking vengono collocate sul perimetro in base al numero disponibile, cercando di mantenerle simmetriche quando possibile. I robot ricevono posizioni iniziali casuali non sovrapposte all'interno dell'area libera della griglia.

Il tempo di percorrenza tra due celle viene stimato tramite distanza di Manhattan divisa per `robot_speed` con l'aggiunta di un ritardo randomico, utilizzata solo all'interno di `Warehouse`, mantenendo il resto del codice indipendente dalla metrica di distanza.

5 Simulator Engine

La classe `simulator` contiene le informazioni riguardanti il proprio stato e il metodo che manda avanti il loop di simulazione.

Attribute	Type	Description
`RNG`	`numpy.Generator`	Seeded RNG for reproducibility
`order_gen_config`	`list[float]`	`[interarrival_time, prob_1_item, geo_param]`
`optimization_enabled`	`bool`	If False, use heuristic policies
`warehouse_status`	`Warehouse`	Physical environment and status of entities
`current_time`	`float`	Current simulation time (minutes)
`future_events`	`PriorityQueue[Event]`	Pending events, keyed by time
`orders_in_system`	`PriorityQueue[Order]`	All received orders; BACKLOG orders surface first
`task_counter`	`int`	Monotonically increasing task ID
`released_tasks`	`PriorityQueue[Task]`	Tasks ready for robot assignment
`scheduled_tasks`	`dict[int, PriorityQueue[Task]]`	Tasks scheduled for each workstation by the optimizer

5.1 Event Loop

Il simulatore esegue un loop DES standard:

1. Pop the earliest event from `event_queue`.
2. Advance `current_time` to the event's timestamp.
3. Dispatch the event to its handler via the dispatch table.
4. Repeat until the queue is empty or the time horizon is reached.

5.2 Order Generation

La generazione degli ordini segue quella usata da [BJS24].

- Con probabilità `PROB_1_ITEM_ORDER`, l'ordine contine un singolo item.
- Altrimenti, il numero di SKU è dato da Geometric$(p) + 2$ con $p =$ `GEO_DIST_PARAM_ORDER`.
- Gli indici delle SKU sono estratta da una normale troncata centrata in $N/2$ e con deviazione standard $N/6$, dove N corrisponde a `NUM_SKUS`.

6 Event Handlers

Ogni `EventType` è associato a una singola funzione handler con firma `handler(event, sim)`. La funzione legge e modifica lo stato della simulazione attraverso l'oggetto `sim`.

Evento	Stato	Responsabilità
`ARRIVAL_ORDER`	**DONE**	Genera un nuovo ordine, lo inserisce in `arrived_orders` e programma il prossimo arrivo. Se non è attivo l'ottimizzatore: assegna l'ordine alla workstation meno occupata, aprendo immediatamente o mettendo in coda.
`OPEN_ORDER`	**DONE**	Imposta lo stato dell'ordine a `OPEN` e lo aggiunge alla lista `open_orders` della workstation. Se la coda dei pod non è piena, attiva `RELEASE_TASK`.
`RELEASE_TASK`	**DONE**	Se non è attivo l'ottimizzatore: chiama `design_tasks_for_ws()` per generare i task, li inserisce in `released_tasks` e aggiorna la lista `released_tasks` della workstation.
`RUN_OPTIMIZER`	**TODO**	Chiama l'ottimizzatore esterno per assegnare gli ordini in backlog e generare i task.
`START_TASK`	**TODO**	Assegna un robot inattivo a un task rilasciato; imposta lo stato del robot e del pod su `BUSY`.
`ARRIVAL_POD_WST`	**TODO**	Gestisce l'arrivo del pod alla workstation; lo aggiunge alla `pod_queue` e avvia il picking se la postazione è libera.
`START_PICKING`	**TODO**	Avvia la sessione di picking; imposta lo stato della workstation su `BUSY` e programma `END_PICKING`.
`END_PICKING`	**TODO**	Completa il picking; decrementa `sku_remaining` per gli ordini coinvolti.
`CLOSE_ORDER`	**TODO**	Imposta lo stato dell'ordine su `CLOSED` quando tutti gli SKU sono stati prelevati; libera lo slot della workstation.
`RETURN_POD`	**TODO**	Riporta il pod alla `home_position`; imposta robot e pod su `IDLE`.

References

[BJS24] Cynthia Barnhart, Alexandre Jacquillat, and Alexandria Schmid. "Robotic warehousing operations: a learn-then-optimize approach to large-scale neighborhood search". In: *ArXiv* abs/2408.16890 (2024). URL: https://api.semanticscholar.org/CorpusID:272310402.